UCS3512 – MACHINE LEARNING LABORATORY

All Four Batches: Question Paper Topics + Separate Python Code

Exam-oriented revision guide

Prepared around the four batch topics in the provided ML Model paper: Heart Disease Prediction, Student Performance Prediction, Glass Type Classification, and Diabetes Prediction.

How to use: Run each batch in a fresh notebook, execute Part 1 first, then run only the later parts required by the question. Replace the CSV filename and target column after checking the dataset columns.

Important: The provided ML Model.pdf is image-based, so the question wording below is organized from the visible batch topics and the task list available in this conversation; it is a study guide, not a verbatim OCR transcription of every printed line.

Before you start: universal exam workflow

- **1. Inspect:** load the dataset, print shape, columns, missing values, and a few rows.
- **2. Identify the target:** classification predicts categories; regression predicts a numerical value.
- **3. Clean and encode:** handle missing values and convert categorical input columns to numeric form.
- **4. Split:** use train_test_split; use stratify=y for classification when every class has enough samples.
- **5. Train the requested model(s):** use the model specified in the question, not necessarily the example model here.
- **6. Evaluate:** classification uses accuracy/precision/recall/F1/confusion matrix; regression uses MAE/MSE/RMSE/R^2.
- **7. Add special requirement:** cross-validation, PCA, bagging, or boosting only if asked.

Target-column warning: never assume the column is called target, Type, Outcome, or G3 without checking df.columns.tolist(). Use the exact column name from your file.

Filename warning: update each pd.read_csv(...) line to the actual file location/name. For the common UCI student-mat dataset, the separator is often a semicolon: pd.read_csv('student-mat.csv', sep=';').

BATCH 1 — Heart Disease Prediction

Problem type: Classification.

Question paper topics / tasks

• Perform exploratory data analysis (EDA) and inspect the heart-disease dataset.

• Visualize numeric features and the target distribution.

• Use a correlation heatmap / correlation-based feature inspection.

• Split data into training and testing sets and train a K-Nearest Neighbours (KNN) classifier.

• Train Support Vector Machine (SVM) models using different kernels (linear, RBF, polynomial).

• Evaluate using accuracy, precision, recall, F1-score, and confusion matrix.

• Use stratified cross-validation.

• Try an AdaBoost model with an SVM base estimator if explicitly required.

Part 1 — imports and dataset loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report

df = pd.read_csv("heart.csv")  # Change filename if needed
print(df.head())
print(df.shape)
print(df.columns.tolist())
print(df.isnull().sum())
```

Part 2 — target, features, and encoding

```
df = df.drop_duplicates()
target = "target"  # CHANGE to the exact target column in your dataset
df = df.dropna(subset=[target])

X = df.drop(columns=[target])
y = df[target]

X = pd.get_dummies(X, drop_first=True)
y = pd.factorize(y)[0]  # works for numeric or text class labels

print("X shape:", X.shape)
print("Classes:", np.unique(y))
```

Part 3 — EDA and target distribution

```
print(df.info())
print(df.describe(include="all"))
print(df[target].value_counts())

df.select_dtypes(include=np.number).hist(figsize=(12, 8))
plt.tight_layout()
plt.show()

sns.countplot(x=df[target])
plt.title("Heart Disease Target Distribution")
plt.show()
```

Part 4 — correlation heatmap

```
numeric_df = df.select_dtypes(include=np.number)
plt.figure(figsize=(11, 7))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

if target in numeric_df.columns:
```

```
    print(numeric_df.corr()[target].abs().sort_values(ascending=False))
```

Part 5 — split data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(X_train.shape, X_test.shape)
```

Part 6 — KNN model and metrics

```
knn = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
knn.fit(X_train, y_train)
pred = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred, average="weighted", zero_division=0))
print("Recall:", recall_score(y_test, pred, average="weighted", zero_division=0))
print("F1:", f1_score(y_test, pred, average="weighted", zero_division=0))
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))
```

Part 7 — SVM kernel comparison

```
for kernel in ["linear", "rbf", "poly"]:
    model = make_pipeline(
        SimpleImputer(strategy="median"),
        StandardScaler(),
        SVC(kernel=kernel)
    )
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print("\\nKernel:", kernel)
    print("Accuracy:", accuracy_score(y_test, pred))
    print("F1:", f1_score(y_test, pred, average="weighted", zero_division=0))
```

Part 8 — stratified 5-fold cross-validation

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(knn, X, y, cv=cv, scoring="accuracy")
print("CV scores:", scores)
print("Mean CV accuracy:", scores.mean())
```

Part 9 — AdaBoost with SVM base estimator (compatibility note)

Some scikit-learn versions do not support passing sample weights through an SVC pipeline. This example preprocesses the data separately and uses SVC directly. AdaBoost requires a base estimator that can learn from sample weights; if this still fails in your environment, show the exact error to your instructor and use the ensemble/model specified in the question.

```
imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()
Xtr = scaler.fit_transform(imputer.fit_transform(X_train))
Xte = scaler.transform(imputer.transform(X_test))

# SVC with probability=True is not needed for this fit; sample_weight is supported by SVC.
# Some combinations/versions can still fail in AdaBoost. Do not use a Pipeline here.
base_svm = SVC(kernel="linear")
ada = AdaBoostClassifier(estimator=base_svm, n_estimators=10, random_state=42)
ada.fit(Xtr, y_train)
pred = ada.predict(Xte)
print("AdaBoost + SVM accuracy:", accuracy_score(y_test, pred))
```

BATCH 2 — Student Performance Prediction

Problem type: Regression when the target is a numeric grade such as G3.

Question paper topics / tasks

• Inspect and visualize the student performance dataset.

• Check missing values, duplicates, summary statistics, and feature correlations.

• Use correlation analysis to inspect potentially useful features.

• Train Linear Regression and Decision Tree Regressor models.

• Evaluate using MAE, MSE, RMSE, and R^2.

• Perform 5-fold cross-validation.

• Implement a Bagging Regressor and compare performance.

Part 1 — imports and loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import BaggingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("student-mat.csv", sep=";")  # use sep=";" for common UCI file
print(df.head())
print(df.shape)
print(df.columns.tolist())
print(df.isnull().sum())
```

Part 2 — target and encoding

```
df = df.drop_duplicates()
target = "G3"  # change if question specifies a different target
df = df.dropna(subset=[target])

X = pd.get_dummies(df.drop(columns=[target]), drop_first=True)
y = df[target]

print(X.shape)
print(y.head())
```

Part 3 — EDA and correlation

```
print(df.info())
print(df.describe(include="all"))
print("Duplicates:", df.duplicated().sum())

sns.histplot(df[target], bins=15, kde=True)
plt.title("Final Grade Distribution")
plt.show()

numeric_df = df.select_dtypes(include=np.number)
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

print(numeric_df.corr()[target].abs().sort_values(ascending=False))
```

Part 4 — split data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Part 5 — Linear Regression

```
lr = make_pipeline(SimpleImputer(strategy="median"), LinearRegression())
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred_lr))
```

```
print("MSE:", mean_squared_error(y_test, pred_lr))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred_lr)))
print("R2:", r2_score(y_test, pred_lr))
```

Part 6 — Decision Tree Regression

```
dt = make_pipeline(
    SimpleImputer(strategy="median"),
    DecisionTreeRegressor(max_depth=5, random_state=42)
)
dt.fit(X_train, y_train)
pred_dt = dt.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred_dt))
print("MSE:", mean_squared_error(y_test, pred_dt))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred_dt)))
print("R2:", r2_score(y_test, pred_dt))
```

Part 7 — compare models

```
results = pd.DataFrame({
    "Model": ["Linear Regression", "Decision Tree"],
    "MAE": [mean_absolute_error(y_test, pred_lr), mean_absolute_error(y_test, pred_dt)],
    "RMSE": [
        np.sqrt(mean_squared_error(y_test, pred_lr)),
        np.sqrt(mean_squared_error(y_test, pred_dt))
    ],
    "R2": [r2_score(y_test, pred_lr), r2_score(y_test, pred_dt)]
})
print(results)
```

Part 8 — 5-fold cross-validation

```
cv = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(lr, X, y, cv=cv, scoring="r2")
print("R2 scores:", scores)
print("Mean R2:", scores.mean())
```

Part 9 — Bagging Regressor

```
imputer = SimpleImputer(strategy="median")
Xtr = imputer.fit_transform(X_train)
Xte = imputer.transform(X_test)

bag = BaggingRegressor(
    estimator=DecisionTreeRegressor(max_depth=5),
    n_estimators=50,
    random_state=42
)
bag.fit(Xtr, y_train)
pred_bag = bag.predict(Xte)

print("MAE:", mean_absolute_error(y_test, pred_bag))
print("MSE:", mean_squared_error(y_test, pred_bag))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred_bag)))
print("R2:", r2_score(y_test, pred_bag))
```

BATCH 3 — Glass Type Classification

Problem type: Classification.

Question paper topics / tasks

• Inspect the glass dataset and perform EDA.

• Plot feature distributions and a correlation heatmap.

• Perform correlation-based feature inspection where appropriate.

• Train Logistic Regression and MLP Classifier models.

• Evaluate using accuracy, precision, recall, F1-score, and confusion matrix.

• Apply PCA retaining approximately 90–95% of variance.

• Implement Bagging with Logistic Regression and/or cross-validation if requested.

Part 1 — imports and loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.decomposition import PCA
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report

df = pd.read_csv("glass.csv")  # change to your actual filename
print(df.head())
print(df.shape)
print(df.columns.tolist())
print(df.isnull().sum())
```

Part 2 — target and feature preparation

```
df = df.drop_duplicates()
target = "Type"  # use exact target name in your dataset
df = df.dropna(subset=[target])

X = df.drop(columns=[target])
# If an ID column exists, remove it if it is only an identifier:
# X = X.drop(columns=["Id"], errors="ignore")
X = pd.get_dummies(X, drop_first=True)
y = pd.factorize(df[target])[0]

print(X.shape, np.unique(y))
```

Part 3 — EDA and correlation heatmap

```
print(df.info())
print(df.describe(include="all"))
print(df[target].value_counts())

df.select_dtypes(include=np.number).hist(figsize=(12, 8))
plt.tight_layout()
plt.show()

numeric_df = df.select_dtypes(include=np.number)
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

Part 4 — split data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Part 5 — Logistic Regression

```
lr = make_pipeline(
    SimpleImputer(strategy="median"),
```

```
    StandardScaler(),
    LogisticRegression(max_iter=2000)
)
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred_lr))
print("Precision:", precision_score(y_test, pred_lr, average="weighted", zero_division=0))
print("Recall:", recall_score(y_test, pred_lr, average="weighted", zero_division=0))
print("F1:", f1_score(y_test, pred_lr, average="weighted", zero_division=0))
print(confusion_matrix(y_test, pred_lr))
print(classification_report(y_test, pred_lr, zero_division=0))
```

Part 6 — MLP Classifier

```
mlp = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(100,), max_iter=1000, random_state=42)
)
mlp.fit(X_train, y_train)
pred_mlp = mlp.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred_mlp))
print("Precision:", precision_score(y_test, pred_mlp, average="weighted", zero_division=0))
print("Recall:", recall_score(y_test, pred_mlp, average="weighted", zero_division=0))
print("F1:", f1_score(y_test, pred_mlp, average="weighted", zero_division=0))
```

Part 7 — PCA retaining 95% variance

```
imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()
Xtr = scaler.fit_transform(imputer.fit_transform(X_train))
Xte = scaler.transform(imputer.transform(X_test))

pca = PCA(n_components=0.95)
Xtr_pca = pca.fit_transform(Xtr)
Xte_pca = pca.transform(Xte)

print("Original features:", X_train.shape[1])
print("PCA features:", Xtr_pca.shape[1])
print("Explained variance:", pca.explained_variance_ratio_.sum())

model_pca = LogisticRegression(max_iter=2000)
model_pca.fit(Xtr_pca, y_train)
pred_pca = model_pca.predict(Xte_pca)
print("PCA accuracy:", accuracy_score(y_test, pred_pca))
```

Part 8 — Bagging with Logistic Regression

```
imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()
Xtr = scaler.fit_transform(imputer.fit_transform(X_train))
Xte = scaler.transform(imputer.transform(X_test))

bag = BaggingClassifier(
    estimator=LogisticRegression(max_iter=2000),
    n_estimators=10,
    random_state=42
)
bag.fit(Xtr, y_train)
pred_bag = bag.predict(Xte)
print("Bagging accuracy:", accuracy_score(y_test, pred_bag))
print(classification_report(y_test, pred_bag, zero_division=0))
```

Part 9 — stratified cross-validation

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(lr, X, y, cv=cv, scoring="accuracy")
print("Scores:", scores)
print("Mean accuracy:", scores.mean())
```

BATCH 4 — Diabetes Prediction

Problem type: Classification.

Question paper topics / tasks

• Inspect and visualize the diabetes dataset.

• Check missing values, summary statistics, and target distribution.

• Scale features and inspect feature correlations.

• Train KNN and Random Forest models with specified hyperparameters.

• Evaluate using classification metrics and a confusion matrix.

• Apply PCA retaining approximately 90–95% of variance.

• Implement Bagging with KNN and cross-validation if requested.

Part 1 — imports and loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, BaggingClassifier
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report

df = pd.read_csv("diabetes.csv")
print(df.head())
print(df.shape)
print(df.columns.tolist())
print(df.isnull().sum())
```

Part 2 — target and features

```
df = df.drop_duplicates()
target = "Outcome"  # common Pima dataset; change to exact target column
df = df.dropna(subset=[target])

X = pd.get_dummies(df.drop(columns=[target]), drop_first=True)
y = pd.factorize(df[target])[0]

print(X.shape, np.unique(y))
```

Part 3 — EDA and correlation heatmap

```
print(df.info())
print(df.describe())
print(df[target].value_counts())

df.select_dtypes(include=np.number).hist(figsize=(12, 8))
plt.tight_layout()
plt.show()

numeric_df = df.select_dtypes(include=np.number)
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

Part 4 — split data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Part 5 — KNN classifier

```
knn = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
knn.fit(X_train, y_train)
```

```
pred_knn = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred_knn))
print("Precision:", precision_score(y_test, pred_knn, average="weighted", zero_division=0))
print("Recall:", recall_score(y_test, pred_knn, average="weighted", zero_division=0))
print("F1:", f1_score(y_test, pred_knn, average="weighted", zero_division=0))
print(confusion_matrix(y_test, pred_knn))
```

Part 6 — Random Forest classifier

```
rf = make_pipeline(
    SimpleImputer(strategy="median"),
    RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred_rf))
print("Precision:", precision_score(y_test, pred_rf, average="weighted", zero_division=0))
print("Recall:", recall_score(y_test, pred_rf, average="weighted", zero_division=0))
print("F1:", f1_score(y_test, pred_rf, average="weighted", zero_division=0))
print(confusion_matrix(y_test, pred_rf))
```

Part 7 — PCA retaining 95% variance

```
imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()
Xtr = scaler.fit_transform(imputer.fit_transform(X_train))
Xte = scaler.transform(imputer.transform(X_test))

pca = PCA(n_components=0.95)
Xtr_pca = pca.fit_transform(Xtr)
Xte_pca = pca.transform(Xte)

print("Original features:", X_train.shape[1])
print("PCA features:", Xtr_pca.shape[1])
print("Explained variance:", pca.explained_variance_ratio_.sum())

knn_pca = KNeighborsClassifier(n_neighbors=5)
knn_pca.fit(Xtr_pca, y_train)
pred_pca = knn_pca.predict(Xte_pca)
print("PCA KNN accuracy:", accuracy_score(y_test, pred_pca))
```

Part 8 — Bagging with KNN

```
imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()
Xtr = scaler.fit_transform(imputer.fit_transform(X_train))
Xte = scaler.transform(imputer.transform(X_test))

bag = BaggingClassifier(
    estimator=KNeighborsClassifier(n_neighbors=5),
    n_estimators=10,
    random_state=42
)
bag.fit(Xtr, y_train)
pred_bag = bag.predict(Xte)

print("Bagging KNN accuracy:", accuracy_score(y_test, pred_bag))
print("Precision:", precision_score(y_test, pred_bag, average="weighted", zero_division=0))
print("Recall:", recall_score(y_test, pred_bag, average="weighted", zero_division=0))
print("F1:", f1_score(y_test, pred_bag, average="weighted", zero_division=0))
```

Part 9 — stratified 5-fold cross-validation

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(knn, X, y, cv=cv, scoring="accuracy")
print("CV scores:", scores)
print("Mean accuracy:", scores.mean())
```

Quick reference — what changes between batches?

Batch	Problem	Target example	Main models	Metrics
1. Heart Disease	Classification	target / output	KNN, SVM, AdaBoost	Accuracy, precision, recall, F1
2. Student Performance	Regression	G3	Linear Regression, Decision Tree, Bagging	MAE, MSE, RMSE, R^2
3. Glass Type	Classification	Type / class	Logistic Regression, MLP, Bagging	Accuracy, precision, recall, F1
4. Diabetes	Classification	Outcome / class	KNN, Random Forest, Bagging	Accuracy, precision, recall, F1

Common troubleshooting

• **KeyError for target:** print(df.columns.tolist() and use the exact column spelling/capitalization.

• **FileNotFoundError:** put the CSV in the notebook's working folder or use the correct file path.

• **Could not convert string to float:** encode categorical input features with pd.get_dummies(X, drop_first=True).

• **NaN error:** use SimpleImputer; fit it on training data and transform test data.

• **Stratified split error:** one or more classes may have too few rows; remove stratify=y or use a suitable split.

• **Bagging estimator error:** newer scikit-learn uses estimator=; older versions may use base_estimator=.

• **AdaBoost + SVM sample_weight error:** avoid wrapping SVC in a Pipeline as the AdaBoost base estimator. Preprocess separately; if it still fails, the installed version/model combination may not support it.

• **Zero values in diabetes data:** some columns may use 0 as a missing-measurement marker, but only replace zeros if the dataset instructions say to do so.

Exam tip: The question's exact requirements take priority. Do not run every section automatically if the examiner asks for only one or two tasks. For PCA, fit scaling and PCA on training data only, then transform the test data.